

Linked Data Structures Templates

Rules Modify outside pointer: use ** or return new root/head. malloc check NULL. pop/remove: 1 success, 0 fail. Delete: save next then free. Queue empty: head=tail=NULL. Tree free: postorder.

Common headers + success style

```
#include <stdio.h>
#include <stdlib.h>
/* return 1 = success, 0 = fail */
```

1.1. Singly Linked List

struct + new node

```
typedef struct node {
    int value;
    struct node *next;
} Node;
Node *new_node(int value)
{
    Node *n = malloc(sizeof *n);
    if (n == NULL) return NULL;
    n->value = value;
    n->next = NULL;
    return n;
}
```

init

```
Node *list_new(void)
{
    return NULL; // empty list head
}
```

insert front / push front

```
void list_push_front(Node **head, int value)
{
    if (*head == NULL) return;
    Node *n = new_node(value);
    if (n == NULL) return;
    n->next = *head;
    *head = n;
}
```

insert back / append

```
void list_append(Node **head, int value)
{
    if (*head == NULL) return;
    Node *n = new_node(value);
    if (n == NULL) return;
    if (*head == NULL) {
        *head = n;
        return;
    }
    Node *curr = *head;
    while (curr->next != NULL)
        curr = curr->next;
    curr->next = n;
}
```

search

```
int list_contains(Node *head, int target)
{
    Node *curr = head;
    while (curr != NULL) {
        if (curr->value == target) return 1;
        curr = curr->next;
    }
    return 0;
}
```

remove first matching value

```
int list_remove(Node **head, int target)
{
    if (*head == NULL || *head == NULL) return 0;
    Node *curr = *head;
    Node *prev = NULL;
    while (curr != NULL) {
        if (curr->value == target) {
            if (prev == NULL) *head = curr->next;
            else prev->next = curr->next;
            free(curr);
            return 1;
        }
        prev = curr;
        curr = curr->next;
    }
    return 0;
}
```

pop front

```
int list_pop_front(Node **head, int *out)
{
    if (*head == NULL || *head == NULL || out == NULL) return 0;
    Node *tmp = *head;
    *out = tmp->value;
    *head = tmp->next;
    free(tmp);
    return 1;
}
```

free

```
void list_free(Node *head)
{
    Node *curr = head;
    while (curr != NULL) {
        Node *next = curr->next;
        free(curr);
        curr = next;
    }
}
```

2.2. Stack using Linked List

struct + init

```
typedef struct stack {
    Node *top;
} Stack;
Stack *stack_new(void)
{
    Stack *s = malloc(sizeof *s);
    if (s == NULL) return NULL;
    s->top = NULL;
    return s;
}
```

push

```
int stack_push(Stack *s, int value)
{
    if (s == NULL) return 0;
    Node *n = new_node(value);
    if (n == NULL) return 0;
    n->next = s->top;
    s->top = n;
    return 1;
}
```

pop

```
int stack_pop(Stack *s, int *out)
{
    if (s == NULL || s->top == NULL || out == NULL)
        return 0;
    *out = s->top->value;
    s->top = s->top->next;
    free(s->top);
    return 1;
}
```

peek + free

```
int stack_peek(Stack *s, int *out)
{
    if (s == NULL || s->top == NULL || out == NULL)
        return 0;
    *out = s->top->value;
    return 1;
}
```

Rule Stack = LIFO. Push/pop at top. Empty stack: top == NULL.

3.3. Queue using Linked List

struct + init

```
typedef struct queue {
    Node *head;
    Node *tail;
} Queue;
Queue *queue_new(void)
{
    Queue *q = malloc(sizeof *q);
    if (q == NULL) return NULL;
    q->head = NULL;
    q->tail = NULL;
    return q;
}
```

enqueue back

```
int enqueue(Queue *q, int value)
{
    if (q == NULL) return 0;
    Node *n = new_node(value);
    if (n == NULL) return 0;
    if (q->tail == NULL) { // empty queue
        q->head = n;
        q->tail = n;
    } else {
        q->tail->next = n;
        q->tail = n;
    }
    return 1;
}
```

dequeue front

```
int dequeue(Queue *q, int *out)
{
    if (q == NULL || q->head == NULL || out == NULL)
        return 0;
    Node *tmp = q->head;
    *out = tmp->value;
    q->head = tmp->next;
    if (q->head == NULL) { // now empty
        q->tail = NULL;
    }
    free(tmp);
    return 1;
}
```

peek + free

```
int queue_peek(Queue *q, int *out)
{
    if (q == NULL || q->head == NULL || out == NULL)
        return 0;
    *out = q->head->value;
    return 1;
}
```

void queue_free(Queue *q)

```
if (q == NULL) return;
list_free(q->head);
free(q);
```

Rule Queue = FIFO. Enqueue at tail; dequeue from head. If empty, both head and tail

are NULL.

4.4. Dequeue using Doubly Linked List

struct + new node

```
typedef struct dnode {
    int value;
    struct dnode *prev;
    struct dnode *next;
} DNode;
typedef struct deque {
    DNode *front;
    DNode *back;
} Deque;
Deque *new_deque(int value)
{
    DNode *n = malloc(sizeof *n);
    if (n == NULL) return NULL;
    n->value = value;
    n->prev = NULL;
    n->next = NULL;
    return n;
}
```

init

```
Deque *deque_new(void)
{
    Deque *dq = malloc(sizeof *dq);
    if (dq == NULL) return NULL;
    dq->front = NULL;
    dq->back = NULL;
    return dq;
}
```

push front

```
int deque_push_front(Deque *dq, int value)
{
    if (dq == NULL) return 0;
    DNode *n = new_dnode(value);
    if (n == NULL) return 0;
    if (dq->front == NULL) {
        dq->front = n;
        dq->back = n;
    } else {
        n->next = dq->front;
        dq->front->prev = n;
        dq->front = n;
    }
    return 1;
}
```

push back

```
int deque_push_back(Deque *dq, int value)
{
    if (dq == NULL) return 0;
    DNode *n = new_dnode(value);
    if (n == NULL) return 0;
    if (dq->back == NULL) {
        dq->front = n;
        dq->back = n;
    } else {
        n->prev = dq->back;
        dq->back->next = n;
        dq->back = n;
    }
    return 1;
}
```

pop front

```
int deque_pop_front(Deque *dq, int *out)
{
    if (dq == NULL || dq->front == NULL || out == NULL)
        return 0;
    DNode *tmp = dq->front;
    *out = tmp->value;
    dq->front = tmp->next;
    if (dq->front == NULL) dq->back = NULL;
    else dq->front->prev = NULL;
    free(tmp);
    return 1;
}
```

pop back

```
int deque_pop_back(Deque *dq, int *out)
{
    if (dq == NULL || dq->back == NULL || out == NULL)
        return 0;
    DNode *tmp = dq->back;
    *out = tmp->value;
    dq->back = tmp->prev;
    if (dq->back == NULL) dq->front = NULL;
    else dq->back->next = NULL;
    free(tmp);
    return 1;
}
```

free

```
void deque_free(Deque *dq)
{
    if (dq == NULL) return;
    DNode *curr = dq->front;
    while (curr != NULL) {
        DNode *next = curr->next;
        free(curr);
        curr = next;
    }
    free(dq);
}
```

Rule Dequeue supports push/pop at both ends. Maintain prev and next. If empty: front=back=NULL.

5.5. Binary Tree

struct + new node

```
typedef struct tree_node {
    int value;
    struct tree_node *left;
    struct tree_node *right;
} TreeNode;
TreeNode *new_tree_node(int value)
{
    TreeNode *n = malloc(sizeof *n);
    if (n == NULL) return NULL;
    n->value = value;
    n->left = NULL;
    n->right = NULL;
    return n;
}
```

init + manual connect

```
TreeNode *tree_new(void)
{
    return NULL; // empty tree root
}
// example: build links manually
root->left = new_tree_node(1);
root->right = new_tree_node(2);
```

preorder traversal

```
void preorder(TreeNode *root)
{
    if (root == NULL) return;
    printf("%d\n", root->value); // root
    preorder(root->left); // left
    preorder(root->right); // right
}
```

inorder traversal

```
void inorder(TreeNode *root)
{
    if (root == NULL) return;
    inorder(root->left); // left
    printf("%d\n", root->value); // root
    inorder(root->right); // right
}
```

postorder traversal + free

```
void postorder(TreeNode *root)
{
    if (root == NULL) return;
    postorder(root->left); // left
    postorder(root->right); // right
    printf("%d\n", root->value); // root
}
void tree_free(TreeNode *root)
{
    if (root == NULL) return;
    tree_free(root->left);
    tree_free(root->right);
    free(root);
    free
}
```

Rule Tree has no ordering rule. Traversal: preorder=root-left-right; inorder=left-root-right; postorder=left-right-root. Free tree by postorder.

6.6. Binary Search Tree BST

Property For every node: left subtree values < root < right subtree values. Inorder prints sorted order.

init

```
TreeNode *bst_new(void)
{
    return NULL; // empty BST root
}
```

insert

```
TreeNode *bst_insert(TreeNode *root, int value)
{
    if (root == NULL)
        return new_tree_node(value);
    if (value < root->value)
        root->left = bst_insert(root->left, value);
    else if (value > root->value)
        root->right = bst_insert(root->right, value);
    return root;
}
```

insert wrapper with double pointer

```
void bst_insert_inplace(TreeNode **root, int value)
{
    if (*root == NULL) return;
    *root = bst_insert(*root, value);
}
```

search

```
int bst_search(TreeNode *root, int target)
{
    if (root == NULL) return 0;
    if (target == root->value) return 1;
    if (target < root->value)
        return bst_search(root->left, target);
    return bst_search(root->right, target);
}
```

find min / max

```
TreeNode *bst_find_min(TreeNode *root)
{
    if (root == NULL) return NULL;
    while (root->left != NULL)
        root = root->left;
    return root;
}
TreeNode *bst_find_max(TreeNode *root)
{
    if (root == NULL) return NULL;
    while (root->right != NULL)
        root = root->right;
    return root;
}
```

delete / remove

```
TreeNode *bst_delete(TreeNode *root, int target)
{
    if (root == NULL) return NULL;
    if (target < root->value) {
        root->left = bst_delete(root->left, target);
    } else if (target > root->value) {
        root->right = bst_delete(root->right, target);
    } else {
        if (root->left == NULL) {
            *root->right = root->right;
            free(root);
            return root->right;
        }
        if (root->right == NULL) {
            *root->left = root->left;
            free(root);
            return root->left;
        }
        // find inorder successor
        TreeNode *succ = bst_find_min(root->right);
        root->value = succ->value;
        root->right = bst_delete(root->right, succ->value);
    }
    return root;
}
```

delete wrapper + sorted output

```
void bst_delete_inplace(TreeNode **root, int target)
{
    if (*root == NULL) return;
    *root = bst_delete(*root, target);
}
void bst_print_sorted(TreeNode *root)
{
    inorder(root); // BST inorder =
    increasing order
}
```

Rule BST delete: 0 child -> free(NULL); 1 child -> return child; 2 children -> replace with inorder successor (min of right subtree), then delete successor.

7.7. High-level File Reading: FILE* + CSV

high-level file I/O headers

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* FILE* high-level stdio. Use fopen/fclose, fscanf/tgets/sscanf/fprintf. */
```

open / close FILE*

```
FILE *fp = fopen("stu_marks.csv", "r");
if (fp == NULL) {
    perror("fopen");
    return 1;
}
// read from fp ...
fclose(fp);
```

average: one integer per line

```
FILE *fp = fopen("stu_marks.csv", "r");
if (fp == NULL) return 1;
int id, mark;
int count = 0;
double sum = 0;
while (fscanf(fp, "%d", &mark) == 1) {
    sum += mark;
    count++;
}
fclose(fp);
printf("%.2f\n", sum / count);
else printf("no marks\n");
```

CSV: id,mark using fscanf

```
FILE *fp = fopen("stu_marks.csv", "r");
if (fp == NULL) return 1;
int id, mark;
int count = 0;
double sum = 0;
while (fscanf(fp, "%d,%d", &id, &mark) == 2) {
    sum += mark;
    // ignore id, use mark
    count++;
}
fclose(fp);
if (count > 0) printf("%.2f\n", sum / count);
```

safer line reading: fgets + sscanf

```
FILE *fp = fopen("stu_marks.csv", "r");
char line[256];
int id, mark;
double sum = 0;
int count = 0;
while (fgets(line, sizeof line, fp) != NULL) {
    if (sscanf(line, "%d,%d", &id, &mark) == 2) {
        sum += mark;
        count++;
    }
}
fclose(fp);
```

read unknown lines with fgets

```
char line[256];
while (fgets(line, sizeof line, fp) != NULL) {
    line[strlen(line, '\n')] = '\0';
    // process line
}
```

write high-level output

```
FILE *out = fopen("out.txt", "w");
if (out == NULL) return 1;
fprintf(out, "average = %.2f\n", avg);
fclose(out);
```

Exam pattern for this question

1. fopen the file with "r".
2. Initialise sum=0 and count=0.
3. Loop while fscanf/fgets succeeds.
4. Add only the mark, then count++.
5. fclose the file.
6. Average = sum/count, but check count > 0.
7. If line is "id,mark", parse two ints and ignore id.

High-level vs low-level

Use FILE* high-level when reading text/CSV lines: fopen, fclose, fscanf, fgets, sscanf, fprintf. Use fd low-level only for OS/system-call questions: open, read, write, close, select, epoll.

8.8. Security: Static vs Shared Libraries

static library vs shared object

Static library (.a): linked into the executable at compile/link time. Code is copied into the binary, so the program can run without the library file at runtime. Larger executable, fewer runtime dependencies/path risks. Shared object (.so): dynamically loaded at runtime. Smaller executable; one library can be shared in memory by many processes. Updates can affect all programs using it.

security concerns with shared objects

Path hijacking: loader may load the wrong .so if search path/LD_LIBRARY_PATH is unsafe. ABI/version mismatch: program expects one interface but runtime .so provides another. Unexpected updates: changing a shared library can change behaviour of many programs. Privilege risk: unsafe library loading is dangerous for setuid/privileged programs. Mitigation: trusted absolute paths, correct permissions, versioning, avoid untrusted env vars.

security traps quick table

Overflow: unbounded input/write overwrites stack/heap. Avoid gets/strcpy/scanf("%s"); use fgets, bounded scanf, size checks. Format string: printf(user) is dangerous; use printf("%s", user). Heap bugs: UAF = access after free; double free corrupts heap; set pointer NULL if reused; check malloc/realloc and size overflow. Command/race: system/popen with user input risks command injection; prefer execv fixed argv. TOCTOU/race: checked state may change before use; protect shared state with locks.

exam answer pattern

Static vs shared: Static is copied into binary at link time; shared .so is loaded at runtime. Shared-object security: mention loader path hijack, malicious replacement, ABI/version mismatch, and updates affecting all users. Mitigation: trusted paths/permissions/versioning.

COMP2017 Back Side - C Pointers, Memory Model, Synchronisation + IPC Pattern: draw ownership first, then code. Return 1 success, 0 fail. Check NULL/malloc/system calls.

9.1. Memory Diagram Method

How to draw C memory diagrams
1. Write regions: text/code, data, bss, heap, stack. 2. Put global/static variables in data/bss. 3. Put each function call as a stack frame. 4. Local variables go inside stack frame. 5. malloc objects go on heap. 6. Pointer variable stores an address; arrow points to target. 7. Array name is not a separate pointer variable; array elements are contiguous. 8. Mark strings with final '\0'. 9. Invalid pointers: wild = uninitialised unknown address; dangling = after free/local return; set reused pointer to NULL. 10. Ownership rule: whoever mallocs must eventually free.
Memory regions quick map
text/code machine instructions data initialised global/static bss zero/uninitialised global/static heap malloc/calloc/realloc objects stack locals, params, return addr
stack vs heap example
<pre>int global = 5; // data static int s; // bss void f(void) { int x = 10; // stack int arr[3]; // stack array int *p = malloc(sizeof *p); // heap int *p = free(p); }</pre>
Drawing pointers
int *p = &x; means p is a stack variable storing address of x. Draw box p with arrow to x. *p means value at target. &p means address of pointer variable itself.
pointer basics
<pre>int x = 5; int *p = &x; // changes x *p = 9; printf("%d\n", x); // 9</pre>
Arrays in memory
int a[3] = {1,2,3}; Draw 3 contiguous boxes: a[0], a[1], a[2]. a in expressions often becomes &a[0], but sizeof(a) is whole array.
10.2. Pointers with Arrays and Strings
array length
<pre>int a[10]; size_t n = sizeof(a) / sizeof(a[0]); // only works in same scope for real array</pre>
array parameter becomes pointer
<pre>void f(int a[]) { sizeof(a); // same as int *a // size of pointer, not array }</pre>
pointer arithmetic
<pre>int a[3] = {10,20,30}; int *p=&a; printf("%d\n", *(p + 1)); // 20 // a[1] == *(a + 1)</pre>
2D array parameter
<pre>void print(int a[][4], int rows) { for (int i = 0; i < rows; i++) for (int j = 0; j < 4; j++) printf("%d ", a[i][j]); }</pre>
pointer array vs array pointer
<pre>int *a[10]; // array of 10 int pointers int **a[10]; // pointer to array of 10 ints</pre>
string array vs string pointer
<pre>char s1[] = "abc"; // mutable array: a b c \0 char *s2 = "abc"; // points to string literal; do not edit s1[0] = 'A'; // ok // s2[0] = 'A'; // undefined behaviour</pre>

safe string input
<pre>char buf[100]; if (fgets(buf, sizeof buf, stdin) != NULL) { buf[strlen(buf, "\n")] = '\0'; }</pre>
copy string with malloc
<pre>char *copy_string(const char *s) { char *c = malloc(strlen(s) + 1); if (c == NULL) return NULL; strcpy(c, s); // copies final \0 too return c; }</pre>
argv template
<pre>int main(int argc, char **argv) { for (int i = 0; i < argc; i++) printf("%s\n", argv[i]); return 0; }</pre>
11.3. Enums, Structs, Bitfields, Unions
enum memory
<pre>enum State { OFF, ON, ERROR = 9 }; enum State s = OFF; // usually stored as int</pre>
struct layout + padding
<pre>typedef struct point { char c; // offset 0, size 1 int x; // padding before x, align to 4 double d; // align to 8 } Point; // sizeof(Point) includes internal + tail padding</pre>
Struct rules: order, alignment, padding
Struct fields are stored in declared order. Each field must start at an address that is a multiple of its alignment. If the current offset is not aligned, insert padding before that field. The final struct size is rounded up to the largest field alignment, so arrays of this struct remain aligned. Use sizeof, not just sum of fields.
manual offset calculation pattern
<pre>// offset = current byte position from struct start // for each field in declared order: // 1. align offset up to field alignment // 2. field_offset = offset // 3. offset += sizeof(field) // after last field: // sizeof(struct) = round up to max alignment</pre>
nested struct + union example
<pre>struct vehicle { uint32_t weight; // offset 0 union { // starts after struct { uint32_t passengers, colour; } car; struct { uint32_t seated; uint8_t wheels; uint32_t standing; } bus; } vehicle; char license; };</pre>
Union rules
All union members start at offset 0 relative to the union. A union's size is the size of its largest member, rounded for alignment. Do not add member sizes together. If a struct contains a union, first find the union field's offset inside the outer struct, then add the chosen member's offset inside the union.
Nested struct rule
For outer.inner.field: offset = offset of inner inside outer + offset of field inside inner. If inner is a union, all union alternatives begin from the same union base offset, but their internal fields still follow struct padding rules.
Array of struct rule
Array elements are stored consecutively. a[i].field offset from a[0] = i * sizeof(struct) + field.offset. Therefore vehicles[1].weight is sizeof(struct vehicle) bytes after vehicles[0].weight.
offset exam answer method
<pre>// 1. Write size/alignment: char=1, uint8_t=1, // uint32_t=4, pointer usually 8 on 64-bit. // 2. Compute each inner struct size first. // 3. Union size = max(member sizes), union // offset shared. // 4. Compute outer struct field offsets in order. // 5. Add tail padding to get sizeof(outer struct). // 6. For array element i: base = i * sizeof(struct). // 7. Final answer = base + field offset.</pre>

struct pointer
<pre>Point p; Point *p = &p; p.x = 1; p->x = 2; // same as (*p).x = 2</pre>
bitfields
<pre>typedef struct flags { unsigned int read : 1; unsigned int write : 1; unsigned int mode : 3; } Flags;</pre>
Bitfield rule
Bitfields pack small integer fields into bits. Layout is implementation-dependent. Good for flags, but not reliable for portable binary file formats or exact cross-platform offsets.
union memory
<pre>typedef union data { int i; float f; char c[4]; } Data; Data d; d.i = 123; // d.i, d.f, d.c share the same address</pre>
12.4. Signals
signal handler
<pre>#include <signal.h> #include <unistd.h> volatile sig_atomic_t stop = 0; void handler(int sig) { stop = 1; // async-signal-safe _style } int main(void) { signal(SIGINT, handler); while (!stop) { write(1, "bye\n", 4); } }</pre>
Common signals
SIGINT Ctrl-C. SIGTERM request terminate. SIGKILL cannot be caught. SIGSEGV invalid memory. SIGCHLD child ended.
send signal
<pre>#include <signal.h> kill(pid, SIGTERM); raise(SIGINT);</pre>
13.5. Processes
fork template
<pre>pid_t pid = fork(); if (pid < 0) { perror("fork"); } else if (pid == 0) { // child exit(0); } else { // parent: pid is child pid wait(NULL); }</pre>
exec template
<pre>execvp("ls", "ls", "-l", NULL); perrot("exec"); // only runs if exec failed exit(1);</pre>
wait status
<pre>int status; pid_t c = wait(&status); if (WIFEXITED(status)) { int code = WEXITSTATUS(status); }</pre>
Process rules
fork duplicates process. Parent and child have separate virtual address spaces. File descriptors are inherited. wait prevents zombies. exec replaces current program.
14.6. IPC: pipes, FIFO, shared memory
pipe parent -> child
<pre>int fd[2]; pipe(fd); // before fork pid_t pid = fork(); if (pid == 0) { close(fd[1]); // child reads char buf[100]; size_t n = read(fd[0], buf, sizeof buf); close(fd[0]); exit(0); } else { close(fd[0]); // parent writes char *msg = "hello"; write(fd[1], msg, strlen(msg) + 1); close(fd[1]); wait(NULL); }</pre>

dup2 pipe into exec
<pre>int fd[2]; pipe(fd); if (fork() > 0) { dup2(fd[1], STDOUT_FILENO); close(fd[0]); close(fd[1]); execlp("ls", "ls", NULL); perror("exec"); exit(1); } close(fd[1]); // parent reads command output from fd[0]</pre>
FIFO named pipe
<pre>mkfifo("fifo", 0666); int fd = open("fifo", O_WRONLY); write(fd, "hi", 3); close(fd); int rd = open("fifo", O_RDONLY); char buf[10]; read(rd, buf, sizeof buf); close(rd);</pre>
shared memory with mmap
<pre>int *shared = mmap(NULL, sizeof *shared, PROT_READ PROT_WRITE, MAP_SHARED MAP_ANONYMOUS, -1, 0); if (shared == MAP_FAILED) exit(1); *shared = 0; if (fork() == 0) { (*shared)++; exit(0); } wait(NULL); munmap(shared, sizeof *shared);</pre>
IPC rules
pipe: related processes, fd[0] read, fd[1] write. Close unused ends. FIFO: named pipe for unrelated processes. Shared memory: fastest, but needs synchronisation.
15.7. Threads: Synchronise Shared Resources
pthread create/join
<pre>#include <pthread.h> void *worker(void *arg) { int *x = arg; // argument passed in printf("%d\n", *x); return NULL; } int main(void) { pthread_t t; int v = 5; pthread_create(&t, NULL, worker, &v); pthread_join(t, NULL); // wait for worker return 0; }</pre>
exam pattern: threads already created
Do not write pthread_create if the question says threads already exist. Instead: identify shared variable(s), choose mutex/cond/sem, wrap every read-modify-write in the same lock, wait with while, signal after changing state, unlock on every path.
shared state struct pattern
<pre>typedef struct shared { int count; // shared variable int ready; // shared condition/state Queue *q; // shared data struct pthread_mutex_t m; // protects all fields above pthread_cond_t cv; // wait for state change } Shared; // init once before workers use it Shared s = {0}; pthread_mutex_init(&s.m, NULL); pthread_cond_init(&s.cv, NULL);</pre>
protect shared resource
<pre>// any thread, after it is already running pthread_mutex_lock(&s->m); // critical section: read/write shared data s->count++; enqueue(s->q, item); or update shared flag pthread_mutex_unlock(&s->m);</pre>
wait until shared condition true
<pre>pthread_mutex_lock(&s->m); while (!s->ready) { // always while, not if pthread_cond_wait(&s->cv, &s->m); } // ready is true and lock is held: use shared data pthread_mutex_unlock(&s->m);</pre>
change state then notify
<pre>pthread_mutex_lock(&s->m); s->ready = 1; condition true pthread_cond_signal(&s->cv); // one waiter // pthread_cond_broadcast(&s->cv); // all waiters pthread_mutex_unlock(&s->m);</pre>

Thread memory rule
Threads share heap, globals and file descriptors; each thread has its own stack/registers. If two threads access the same data and at least one writes, protect it. mutex protects data; condition variable waits for a state; semaphore counts resources.
16.8. Applying Synchronisation Primitives
How to choose primitive
Mutex: protect one critical section. Semaphore: count available resources or limit capacity. Condition variable: sleep until a condition becomes true. Atomic/reduction: avoid shared write by using local results, combine later.
mutex critical section
<pre>pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; int counter = 0; void *worker(void *arg) { pthread_mutex_lock(&lock); counter++; // not atomic without lock pthread_mutex_unlock(&lock); return NULL; }</pre>
semaphore template
<pre>#include <semaphore.h> sem_t slots; sem_init(&slots, 0, 3); // 3 resources sem_wait(&slots); // take one, may block // use resource sem_post(&slots); // release one sem_destroy(&slots);</pre>
condition variable wait
<pre>pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; pthread_cond_t cv = PTHREAD_COND_INITIALIZER; int ready = 0; pthread_mutex_lock(&m); while (!ready) { // always while, not if pthread_cond_wait(&cv, &m); } // condition true, do work pthread_mutex_unlock(&m);</pre>
condition variable signal
<pre>pthread_mutex_lock(&m); ready = 1; pthread_cond_signal(&cv); // or broadcast pthread_mutex_unlock(&m);</pre>
producer-consumer: shared buffer state
<pre>typedef struct buffer { Queue *q; int capacity; pthread_mutex_t m; pthread_cond_t not_empty; pthread_cond_t not_full; } Buffer;</pre>
producer: wait if full, then signal consumer
<pre>void produce(Buffer *b, Item item) { pthread_mutex_lock(&b->m); while (queue_size(b->q) == b->capacity) pthread_cond_wait(&b->not_full, &b->m); enqueue(b->q, item); // shared update pthread_cond_signal(&b->not_empty); pthread_mutex_unlock(&b->m); }</pre>
consumer: wait if empty, then signal producer
<pre>int consume(Buffer *b, Item *out) { pthread_mutex_lock(&b->m); while (queue_empty(b->q)) pthread_cond_wait(&b->not_empty, &b->m); *out = dequeue(b->q); // shared update pthread_cond_signal(&b->not_full); pthread_mutex_unlock(&b->m); return 1; }</pre>
producer-consumer exam rules
Use one mutex to protect the queue and count/capacity. Use while around cond_wait. Producer waits on not_full and signals not_empty. Consumer waits on not_empty and signals not_full. Signal after changing the shared state, while still holding the mutex.

parallel reduction pattern
<pre>typedef struct arg { int lo, hi; int *a; long sum; } Arg; void *sum_worker(void *p) { Arg *a = p; long local = 0; for (int i = x->lo; i < x->hi; i++) local += a[i]; // each thread writes x->sum = local; return NULL; } // join all, then total += args[i].sum</pre>
17.9. Deadlock Avoidance Templates
Deadlock rule checklist
Deadlock needs: mutual exclusion + hold-and-wait + no preemption + circular wait. Break circular wait by always locking in one global order. Keep critical sections short. Never wait on condition while holding unrelated locks.
lock ordering pattern
Always acquire locks in the same order everywhere. <pre>pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER; pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER; void safe_work(void) { pthread_mutex_lock(&A); // lock lower/order-first pthread_mutex_lock(&B); // critical section using A and B pthread_mutex_unlock(&B); // unlock reverse order pthread_mutex_unlock(&A); }</pre>
function pointer template
<pre>int add(int a, int b) { return a + b; } // function pointer variable int (*fp)(int, int) = add; int result = fp(2, 3); // calls add // function pointer parameter = callback int apply(int x, int y, int (*op)(int, int)) { return op(x, y); }</pre>
Function pointer rule
Pattern: return_type (*name)(arg_types). Parentheses around *name are required. int *f(int) means function returning int*. Use for callbacks, qsort comparators, dispatch tables.
Deadlock exam traps
Do not lock A then B in one thread and B then A in another. Do not return early while holding a mutex. Cond wait releases only its own mutex, not other locks. Use while for condition wait.
18.10. Process Two-way Communication
two pipes: parent <-> child
<pre>int p2c[2], c2p[2]; pipe(p2c); pipe(c2p); // before fork pid_t pid = fork(); if (pid == 0) { close(p2c[1]); // child reads from parent close(c2p[0]); // child writes to parent enqueue(b->q, item); update pthread_cond_signal(&b->not_empty); pthread_mutex_unlock(&b->m); }</pre>
consumer: wait if empty, then signal producer
<pre>int consume(Buffer *b, Item *out) { pthread_mutex_lock(&b->m); while (queue_empty(b->q)) pthread_cond_wait(&b->not_empty, &b->m); *out = dequeue(b->q); // shared update pthread_cond_signal(&b->not_full); pthread_mutex_unlock(&b->m); return 1; }</pre>
producer-consumer exam rules
Two pipes for full duplex: p2c and c2p. Close unused ends immediately. Close write end when done so peer sees EOF. Avoid both sides reading first. Structs: read sizeof struct; strings: include '\0' or send length.